■ Lab 15 — Advanced Terraform & CI/CD (AWS CloudShell)

■ Objectifs du Lab

- Utiliser le **dependency lock file** (`terraform.lock.hcl`)
- Renommer avec le bloc `moved` sans destroy/recreate
- Valider avec le bloc `check` (assertions post-apply)
- Gérer les **secrets** de manière sécurisée
- Utiliser `terraform plan -out=planfile` + `terraform apply planfile`
- Mettre en place un pipeline **CI/CD GitHub Actions**

■ Étape 1 — Préparer l'environnement

```
bash $HOME/install-terraform.sh
cd $HOME/terraform-training/lab15-advanced
```

Le reste des instructions est identique à la version principale (README.md).

Utilisez `nano` pour éditer les fichiers dans CloudShell.

■ Étape 2 — Cloner le repo GitHub pour le CI/CD

Pour la partie CI/CD, vous avez besoin d'accès à votre repo GitHub depuis CloudShell :

```
# Configurer git
git config --global user.email "<votre-email>"
git config --global user.name "<votre-prenom>"

# Cloner votre repo
git clone https://<votre-token>@github.com/<votre-compte>/tf-training-<votre-prenom>.git
cd tf-training-<votre-prenom>
```

■ *Remplacez `` par votre Personal Access Token GitHub (Settings → Developer settings → Tokens).*

■ Étape 3 — Créer le workflow GitHub Actions

```
mkdir -p .github/workflows

cat > .github/workflows/terraform.yml << 'EOF'
name: Terraform CI/CD

on:
  push:
    branches: [ main ]
    paths: [ 'labs/lab15-advanced/**' ]
  pull_request:
    branches: [ main ]
    paths: [ 'labs/lab15-advanced/**' ]
  workflow_dispatch:
    inputs:
      action:
        description: 'Action à exécuter'
        required: true
        default: 'plan'
        type: choice
        options: [ plan, apply, destroy ]

env:
  TF_WORKING_DIR: labs/lab15-advanced
  TF_VAR_username: ${{ github.actor }}

jobs:
  terraform:
    name: Terraform ${{ github.event.inputs.action || 'plan' }}
    runs-on: ubuntu-latest
    defaults:
```

```
      run:
        working-directory: ${{ env.TF_WORKING_DIR }}
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.15.5"

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: eu-west-1

      - name: Terraform Init
        run: terraform init

      - name: Terraform Format Check
        run: terraform fmt -check -recursive

      - name: Terraform Validate
        run: terraform validate

      - name: Terraform Plan
        run: terraform plan -var="environment=dev" -out=tfplan
        if: github.event_name == 'pull_request' || github.event.inputs.action == 'plan' || github.event_

      - name: Upload Plan
        uses: actions/upload-artifact@v4
        with:
          name: tfplan
          path: ${{ env.TF_WORKING_DIR }}/tfplan
          retention-days: 1
        if: github.event_name == 'pull_request' || github.event.inputs.action == 'plan' || github.event_

      - name: Terraform Apply
        run: terraform apply tfplan
        if: github.event.inputs.action == 'apply'

      - name: Terraform Destroy
        run: |
          terraform plan -destroy -var="environment=dev" -out=tfplan-destroy
          terraform apply tfplan-destroy
        if: github.event.inputs.action == 'destroy'
EOF
```

```
git add .
git commit -m "feat: lab15 advanced terraform + CI/CD"
git push origin main
```

■ Validation du Lab

#	Critère	Validé
1	`.terraform.lock.hcl` présent et commité	■
2	`terraform plan -out=tfplan` + `terraform apply tfplan` fonctionne	■
3	Bloc `moved` renomme sans recréer (`0 to destroy`)	■
4	Bloc `check` valide l'état après apply	■
5	Secrets AWS dans GitHub Actions Secrets	■
6	Pipeline GitHub Actions se déclenche sur push	■
7	Apply et Destroy via `workflow_dispatch` fonctionnent	■

■ **Fin du Lab 15 !**